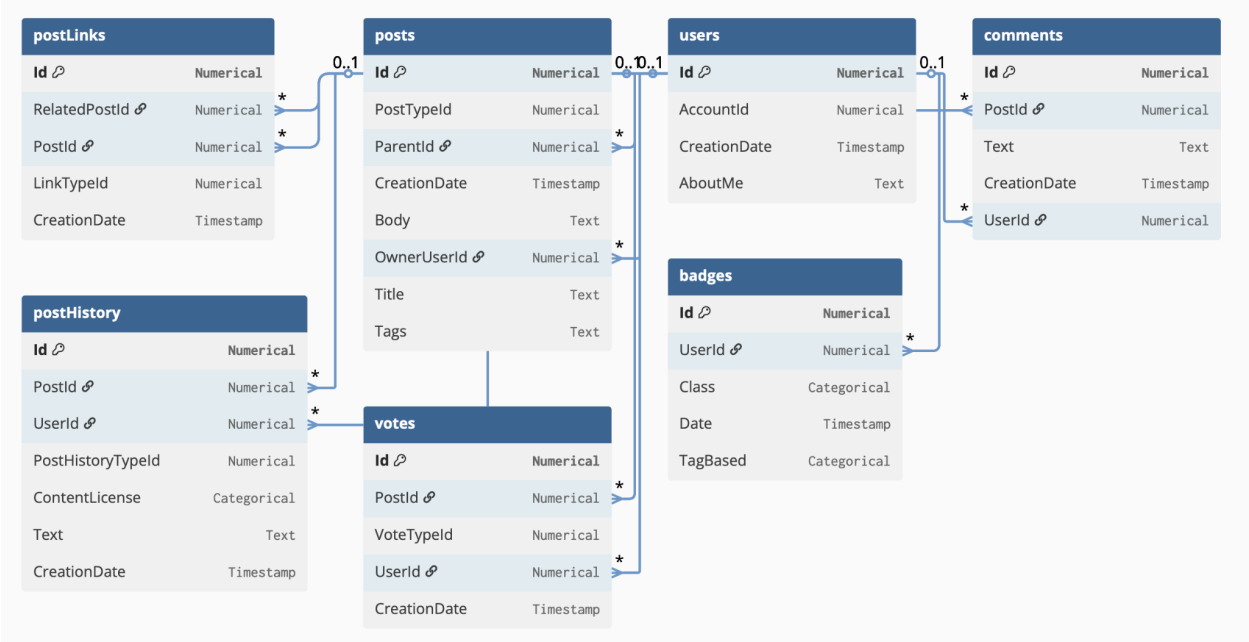
StructML (236605) Assignment 1

Feature Extraction

In this exercise, we will focus on extracting tabular and graph features and using them in a prediction task.

Database: As our database, we will use rel-stack, from the relbench dataset suite.
The database contains information about users, posts and comments in stack overflow, spanning 7 tables. The database schema:



Source: https://relbench.stanford.edu/datasets/rel-stack/.

Prediction task: As the prediction task, we will use `user-engagement`, defined over the users table. The goal is to predict for each user if they will make any votes, posts, or comments in the next 3 months.

Submission Guidelines

Submission is in **pairs** only. Your submission should include a zip of the following:

1. A **report** (pdf file) containing your results and the answers to the questions in this assignment (in order). Note that the last two questions are **summary questions**; it may be useful to populate your response to them throughout your work.
2. LLM/AI usage: you may use AI tools as much as you like. **You will need to report your usage of LLMs in detail** (see last question in this exercise).
3. All of the code you used, written in python, and specifically, a jupyter notebook containing all the results included in your report (graphs, training measurements, etc.) and the code used to generate them. You may use .py files imported into the notebook,

for convenience and readability of your code. Export the notebook (including the results of running it) to html format using the following command:
`jupyter nbconvert --to html your_notebook_with_results.ipynb`

Part 0: Setup

Work environment:
For this, and the next assignments in the course, you have been given access permissions to the Technion CS Lambda cluster. Instructions on how to work with it are available here: https://aihpc.cs.technion.ac.il/lambda/
You may use it, or any other computing resources you have (including your laptop).

Python environment: Use the attached yml file to set up a conda environment. Alternatively, manually install the packages.

Downloading the data: Load the dataset based on the instructions on https://relbench.stanford.edu/start/. Specifically:
To get the full database, use:

```
from relbench.base import Table, Database, Dataset, EntityTask
from relbench.datasets import get_dataset
from relbench.tasks import get_task

dataset = get_dataset("rel-stack", download=True)
db = dataset.get_db()
```

Accessing a table can be done by:
`db.table_dict[table_name].df`

The task **contains 3 tables** which are subsets **of the target table.** In our case, the target table contains the user id and the binary label (“contribution”). The division to train, val, and test sets was done by timestamp. Use this split in your exercise to train and evaluate your models.
To get the division to train/val/test of the **target table**:

```
task = get_task("rel-stack", "user-engagement", download=True)
train_table = task.get_table("train")
val_table = task.get_table("val")
test_table = task.get_table("test")
```

[Edited:] Note that these tables contain only a subset of the attributes in the users table (timestamp, user id and label).

Part 1: Tabular Feature Extraction

In this part you will extract tabular features automatically as described in module 2 (“Features from Databases”).

1. Suggest and implement 3 join-aggregate features that you think can help in the prediction task. What are they? How do you hypothesize that they are related to the prediction task?
2. Implement an automatic join-aggregate feature generation algorithm:
    a. Given an input parameter `max_depth`, perform all joins of up to `max_depth` hops. For example, a join path with a single hop: target ⋊ users. A join path with 2 hops: target ⋊users⋈ posts.
    [Edited:] For joining the target table to the others, use left join to keep all rows of the target table. For all other joins, use a natural join. For more information on types of join see here:
    https://www.atlassian.com/data/sql/sql-join-types-explained-visually.
    [Edited:] Maintaining the temporal constraint:
    Each prediction for a tuple x of the target table should be done based only on tuples that predated the timestamp of x.
    To maintain this constraint **for the train split**, after every join, filter out any tuples whose timestamp is after the timestamp of the target tuple.
    For the **validation split**, filter out any tuples from all tables whose timestamp is after the global dataset.val_timestamp (2020-10-01 00:00:00).
    For the **test split**, there is no need to filter anything, since dataset.get_db already returns only tuples before the dataset.test_timestamp (2021-01-01 00:00:00).
    b. For each joined table achieved this way, for each of the columns in the joined table, create aggregated features using aggregations that match its type.
        i. For numeric columns, use mean, count and sum.
        ii. For categorical/text columns, use count.
        iii. For timestamp columns, use min and max.
    c. Run this algorithm with `max_depth`=2. Collect the original target table and the new features to a single `pandas` dataframe (we will refer to it as `feature_matrix` for the rest of the exercise).
3. How many new features would be created for `max_depth` in {1,2,3}? Explain your answer.

The resulting table:

4. What is the shape of the `feature_matrix` dataframe (how many rows, and how many columns)? Explain how it differs from the original target table and why.
5. Give examples of 3 of the newly extracted features (that were not part of the original table), and explain their meaning (what they represent).
6. Are there any missing values in the extracted features, and why?

Next, train and compare the following classification models over the feature matrix. For reference, note that for this task, using only the users table (without additional features), it is possible to achieve an AUC of 0.65 on the validation set, as was done in the relbench paper.

- `DecisionTreeClassifier`
- `XGBoost`
- `Logistic regression`
- A simple NN model of your choosing, implemented in `PyTorch`. Describe the architecture and training parameters in your report.

7. To compare the models, fill the following table with the evaluation measures of the best performance on the train and validation sets, and add it to your report. If you experimented with different variations of each model for hyperparameter tuning, the table should contain the best performing variation, and a full table of all variations (and their differences) should be included separately. In your report, explain the hyperparameter search you conducted (what hyperparameters were changed, and what values for each one).

	Accuracy train/val	ROC AUC train/val	Precision train/val	Recall Train/val	F1 train/val
Decision Tree					
XGBoost					
Logistic Regression					
Your custom NN					

8. Explain the meaning of each evaluation measure (accuracy, AUROC, precision, recall, F1). Explain the tradeoffs between the measures. Which ones do you think will be most helpful in our prediction task, considering the balance between classes?

9. Discuss the results. Which models were better on the training set? And on the validation set? What might be the reasons for these advantages, based on the mechanisms behind each model, and each evaluation measure? How does the choice of the schema and learning task affect the results?

Each of the prebuilt models (Decision Tree, XGBoost, Logistic Regression) has an attribute which holds a weight for each input feature. [Edited:] For decision tree and XGBoost this is called `feature_importances_`, and for Logistic Regression, the `coef_` attribute should be used. Using this attribute for each of the three models:

10. Explain how the feature importance is computed in each of the models.

11. Compare the top 10 features with highest importance scores for each of the models. Add the lists of top 10 features of each model to your report. Discuss the commonalities and the differences in the lists of top features for each model. What may be the reasons for the differences?

Part 2: Graph Feature Extraction

Note: the material for this part of the exercise is covered in module 2 and tutorial 3.

Using `pandas` and `networkx`, construct a graph representing the database, where each tuple is represented by a node, and the edges are defined by PK-FK connections.

For this exercise, we are only interested in features capturing the graph structure, so there is no need to encode tuple features into the graph. Therefore, the representation of each node is by type (table) and ID (index). For example, a user tuple with index 123 should have the following unique representation in the graph: ("user", 123), as seen in tutorial 3.

To map the DB into a graph, create a node for each tuple and an edge for each FK-PK connection.

[Edited] Temporal constraints:

For simplicity, we will require to enforce only the global temporal constraints. Building the graph for the train split should only include tuples from **before dataset.val_timestamp** (2020-10-01 00:00:00). Building the graph for the validation split should include tuples **up to and including dataset.val_timestamp** (2020-10-01 00:00:00). Building the graph for the test should include all tuples.

12. Compute the following features for each node (tuple). Add the following features, which are based on the graph structure, to the table you got in question 2.
    a. Degree
    b. WL coloring (the "color" id we get from applying K iterations of Weisfeiler-Lehman), for K in [Edited:] {2,3,5}. Note: You do not need to run this until mathematical convergence, as the database graph size causes memory issues.
    c. For each of the following graphlet structures, count the number of times a user participates in such a graphlet:
        i. A cycle of the form: user-comment-post-user (How many times did the user comment on their own post?)

ii. A cycle of the form: user-post-postLinks-post-user (How many times did the user have two linked posts?)

13. Train and compare the models again and report the performance measures in a table such as the one given in question 7.

14. As in question 10, use the `feature_importances_` attribute of each trained model to compare the top 10 features of the models. Which models (if any) used the features extracted from the graph? Which of the features extracted from graphs were most helpful in the prediction? What may be the reasons for these results?

15. **Summary** question: write a conclusion for your report. In your answer:

    a. Describe the steps you took and what you learned in each of them.

    b. What were the advantages and disadvantages of using automated feature extraction (as done in question 2), compared to manual feature extraction (as in question 1)?

    c. What were the advantages and disadvantages of using the features extracted from the graph, compared to only tabular features?

16. AI usage: add a paragraph to your report, describing in detail **where** you used AI tools LLMs (such as ChapGPT, Gemini, etc.). Explain your **critical validation process**—how did you catch hallucinations or optimize the AI's suggestions for the specific tasks? Additionally, estimate the **percentage** of the total workflow in the assignment that was assisted by AI tools. **You can use AI as much as you want, as long as you understand what it did, validate that it was correct, and report it.**

Good Luck!